

FALCON-1 PoC

기술 분석 보고서 (Tech Analysis Report)

영역 H — Grasp Planning (6종)

[accent : BROWN]

프로젝트 : FALCON-1 (Flow-Assisted Logistics & Collaborative Operation Network — One)

팀 : NERDLABS | 버전 : v0.1 | 작성일 : 2026-04-30

작성자 : 이건희 (NERDLABS 팀장) | 검토 : CTO 페르소나

보고서 BLUF

BLUF FALCON-1 Grasp Planning 영역 6종 분석 결과: Hand-coded grasp = MUST (v1.0 PoC 핵심, AprilTag + wrist D405 fine-tuning 결합), GG-CNN = SHOULD (단순 모델 baseline), Contact-GraspNet · GraspNet-1Billion = v1.1 (학습 기반 grasp), AnyGrasp = v1.2+ (동적 환경), Grasplt! = 비권장 (legacy). v1.0 인테리어 마감 작업장의 4종 공구 (driver · hammer · wrench · 4번째)에는 hand-coded approach + Tag-anchored grasp pose가 가장 reliable + reproducible. 학습 기반 grasp는 v1.1 새 공구 추가 narrative.

본 보고서는 영역 D(MoveIt 2 + Pilz) + E(AprilTag · YOLOv11) + F(librealsense2 · Open3D · FoundationPose)을 전제로 한다. AprilTag(E7) 6D pose가 grasp pose anchor 역할, wrist D405 close-range RGB-D가 fine-tuning 보강.

본 영역의 accent 색상은 BROWN (도구·잡기 의미)을 사용한다.

영역 H — 6종 요약 매트릭스

#	기술	방식	정확도	처리 위치	권장도	적용
H1	Hand-coded grasp	AprilTag + offset	±2 mm	RPi5 CPU	★★★★★	MUST (v1.0)
H2	GG-CNN	RGB-D → grasp heatmap	70-85% 성공률	L1 GPU	★★★	SHOULD baseline
H3	Contact-GraspNet	Point cloud → 6D grasp	85-92% 성공률	BRAVO GPU	★★★	v1.1 학습 기반
H4	GraspNet-1Billion	대규모 데이터 학습	92-95% 성공률	BRAVO GPU	★★★	v1.1 SOTA
H5	AnyGrasp	Dynamic + temporal	95%+ (동적)	BRAVO GPU	★★	v1.2+
H6	Grasplt!	시뮬레이션 grasp	—	L1 CPU	★	비권장 (legacy)

H1. Hand-coded grasp (AprilTag-anchored)

BLUF ★★★★★ MUST 채택. v1.0 FALCON-1 PoC 핵심 grasp 전략 — AprilTag 6D pose에서 공구별 hand-coded grasp offset(SE(3)) 적용, wrist D405 close-range RGB-D로 fine-tuning. 학습 데이터 불필요 + reproducible + 100% 디버깅 가능. 4종 공구 각각 고정 grasp pose 정의로 PoC 신뢰성 보장. 학회 paper에는 'AprilTag-anchored Hand-coded baseline' 으로 표기 가능.

1. 개요

항목	내용
방식	AprilTag 6D pose + 공구별 grasp offset (hand-tuned)
라이선스	자체 작성 (BSD)
정확도	AprilTag(E7) $\pm 1\text{mm}$ + grasp offset $\pm 1\text{mm}$ = $\pm 2\text{ mm}$
성공률 (목표)	$\geq 95\%$ (Sprint 5 burn-in 측정)
학습 필요	✗(튜닝만)
디버깅	✔모든 변환 가시화 가능
처리 위치	RPi5 CPU (수십 ms 미만)

2. 적용 방법

2.1 공구별 grasp offset 정의 (FALCON-1)

공구	Tag ID	Tag 부착 위치	grasp offset (Tag → TCP)
driver	0	손잡이 측면	x: +0.05m (앞으로 5cm), y: 0, z: +0.02m (위로 2cm), rot: y축 +90°
hammer	1	손잡이 측면	x: +0.07m, y: 0, z: +0.03m, rot: y축 +90°

공구	Tag ID	Tag 부착 위치	grasp offset (Tag → TCP)
wrench_14mm	2	손잡이 끝	x: +0.06m, y: 0, z: +0.02m, rot: y축 +90°
(예비 4번째)	—	—	Sprint 4 결정

2.2 ROS 2 노드 구현 (Python)

```
# hand_coded_grasp_node.py
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped, TransformStamped
from apriltag_msgs.msg import AprilTagDetectionArray
import tf2_ros
from tf2_geometry_msgs import do_transform_pose
import numpy as np
from scipy.spatial.transform import Rotation as R

# 공구별 grasp offset (Tag frame → TCP frame)
GRASP_OFFSETS = {
    'driver':      {'xyz': [0.05, 0, 0.02], 'rpy': [0, np.pi/2, 0]},
    'hammer':      {'xyz': [0.07, 0, 0.03], 'rpy': [0, np.pi/2, 0]},
    'wrench_14mm': {'xyz': [0.06, 0, 0.02], 'rpy': [0, np.pi/2, 0]},
}

TAG_TO_TOOL = {0: 'driver', 1: 'hammer', 2: 'wrench_14mm'}

class HandCodedGraspNode(Node):
    def __init__(self):
        super().__init__('hand_coded_grasp')
        self.tf_buf = tf2_ros.Buffer()
        self.tf_listener = tf2_ros.TransformListener(self.tf_buf, self)

        self.sub = self.create_subscription(
            AprilTagDetectionArray, '/detections',
            self.cb_detections, 10)
        self.pub = self.create_publisher(
            PoseStamped, '/grasp/target_pose', 10)

    def cb_detections(self, msg):
        for det in msg.detections:
            tool = TAG_TO_TOOL.get(det.id)
            if tool is None: continue

            offset = GRASP_OFFSETS[tool]
            # AprilTag pose → grasp pose (offset 적용)
            tag_pose = det.pose.pose # base_link frame
            grasp_pose = self.apply_offset(tag_pose, offset)

            ps = PoseStamped()
            ps.header.frame_id = 'base_link'
```

```

        ps.header.stamp = self.get_clock().now().to_msg()
        ps.pose = grasp_pose
        self.pub.publish(ps)

    def apply_offset(self, tag_pose, offset):
        # tag_pose: geometry_msgs/Pose
        # offset: {'xyz': [x,y,z], 'rpy': [r,p,y]}
        # → SE(3) 변환 후 base_link frame 기준 grasp pose 반환
        # (구현 생략 - TF + scipy.spatial.transform 활용)
        ...

```

2.3 wrist D405 fine-tuning (Sprint 4 검토)

- AprilTag 광역 grasp pose → approach 단계 (Pilz LIN 10cm 위)
- Approach 후 wrist D405 가 close-range (5-10cm) 공구 표면 RGB-D 측정
- Open3D RANSAC plane fit 으로 공구 표면 normal 정밀 추정
- Hand-coded grasp offset 에 $\pm 2\text{mm}$ 보정 적용
- 최종 grasp 시 정확도 $\pm 1\text{mm}$ 가능 (목표는 $\pm 2\text{mm}$ 로 충분)

2.4 Pick-and-Place 통합 (D3 MTC와 결합)

```

# moveit_task_constructor stage 결합 (영역 D §D3)
Task task;

// Stage: grasp pose 생성
auto grasp_gen = std::make_unique<stages::GeneratePose>("grasp_pose");
grasp_gen->setMonitoredStage(...);

// Hand-coded grasp pose 주입 (AprilTag + offset)
geometry_msgs::msg::PoseStamped grasp_pose = compute_hand_coded_grasp("driver");
grasp_gen->setPose(grasp_pose);
task.add(std::move(grasp_gen));

// Pick stage (Pilz LIN 10cm approach + close + lift)
auto pick = std::make_unique<stages::Pick>("pick", std::move(grasp_gen));
pick->setApproach(0.10, geometry_msgs::Vector3(0,0,-1));
pick->setLift(0.05, geometry_msgs::Vector3(0,0,1));
task.add(std::move(pick));

```

3. 하드웨어 필요·제약

항목	요구	FALCON-1 적합성
AprilTag (E7)	공구 부착 + 캘리브레이션	✓ Sprint 2

항목	요구	FALCON-1 적합성
wrist D405 (F1)	fine-tuning 시	✓TUF Dash F15 직결
CPU	<1% (TF + offset 계산)	✓RPi5
GPU	불필요	✓
RAM	수 MB	✓
Movelt 2 (D1)	필수	✓

4. 장점

1. 학습 데이터 불필요 — Sprint 3 데이터 수집·라벨링 부담 없음
2. Reproducible — 같은 입력 → 같은 출력 100% 보장 (Pilz LIN 과 동일)
3. 디버깅 100% — 모든 변환 (tag → grasp) 가시화
4. RPi5 CPU < 1% 부하
5. AprilTag 6D pose 정확도($\pm 1\text{mm}$)가 grasp 정확도 결정 — 검증 가능
6. v1.0 PoC 4 종 공구는 hand-tuning 으로 충분 — 학습 모델 overengineering
7. 학회 paper 에 'AprilTag-anchored baseline' 으로 활용
8. v1.1 학습 기반 grasp(H3) 도입 시 baseline 으로 비교 가능

5. 단점·한계

9. Tag 부착 의무 — 자연 물체 grasp 불가
10. 새 공구 추가 시 hand-tuning — Sprint 3 에 4 종 × 30 분 = 2 시간 정도
11. 물체 형상 변동 시 실패 — 같은 'driver'라도 모델 다르면 offset 재정의
12. Tag 가림 시 실패 — wrist 카메라가 Tag 보이지 않으면 grasp 시작 불가
13. 학회 narrative 약점 — 'hand-coded' 단어 자체가 과거 기술 인상
14. v1.1 narrative 위해 H3-H4 학습 모델로 전환 의무

6. 대안 비교

항목	Hand-coded (H1)	GG-CNN (H2)	Contact-GraspNet (H3)	AnyGrasp (H5)
방식	AprilTag + offset	RGB-D heatmap	Point cloud → 6D	동적 + temporal
학습 필요	✗	✓데이터 5K	✓데이터 100K+	✓대규모
성공률	95%+ (목표)	70-85%	85-92%	95%+ (동적)
디버깅	✓100%	약함 (CNN 흑상자)	약함	약함
FALCON-1 v1.0	✓MUST	Sprint 4 baseline	v1.1	v1.2+

7. FALCON-1 적용 결정

결정 항목	값
채택 여부	MUST 채택
도입 시점	Sprint 3 (Single Mission v0)
처리 위치	RPi5 CPU
공구 4종 정의	Sprint 3 hand-tuning (각 30분)
fine-tuning 보강	Sprint 4 wrist D405 close-range (선택)
MTC 통합	GeneratePose stage 주입
목표 성공률	≥ 95% (Sprint 5 burn-in 100회)
주요 의존성	E7 AprilTag, F1 D405, D1 MoveIt 2, D2 Pilz

8. 참고

- FALCON-1 자체 작성 (BSD 라이선스)

- AprilTag 부착 + 캘리브레이션 가이드 : 영역 E §E7
- MoveIt MTC GeneratePose :
https://moveit.picknik.ai/main/doc/examples/moveit_task_constructor/moveit_task_constructor_tutorial.html

H2. GG-CNN (Generative Grasping CNN)

BLUF ★★★ SHOULD baseline (Sprint 4 검토). 단순 CNN이 RGB-D depth → grasp heatmap (x, y, θ , w) 출력, 학습 데이터 5K~10K로 70-85% 성공률 가능. v1.0 PoC에서는 Hand-coded(H1)가 우월하나 학회 paper baseline 비교용으로 가치. RTX 3060에서 50+ FPS, 가벼움.

1. 개요

항목	내용
풀네임	Generative Grasping Convolutional Neural Network
출시	2018년
라이선스	BSD
관리	QUT (Douglas Morrison)
입력	depth image (또는 RGB-D)
출력	grasp heatmap (x, y, theta, width)
학습 데이터	Cornell Grasp dataset (5K) 또는 Jacquard (50K)
속도 RTX 3060	50-80 FPS
성공률	70-85% (Cornell dataset 기준)

2. FALCON-1 사용 시나리오

- v1.0 학회 paper baseline — Hand-coded vs GG-CNN 성공률 비교
- Sprint 4 시간 여유 시 Cornell dataset pretrained 모델 onboard 추론 시도
- 실 데이터 fine-tuning 부담 작음 (5K 로 충분)

3. 하드웨어 필요·제약

항목	요구	FALCON-1 적합성
GPU	RTX 3060 6GB	✔TUF, BRAVO 모두
VRAM	1-2 GB	✔
입력	depth image (D405 OK)	✔
속도	50+ FPS	✔실시간
학습 시간 (Cornell)	1-2h (RTX 5090)	✔

4. 장점·단점 요약

장점 :

- 단순 CNN 빠름
- Cornell pretrained 모델 즉시 사용
- BSD 라이선스
- 학회 paper baseline 표준

단점 :

- 성공률 70-85% — Hand-coded 95% 미달
- 2D heatmap 한계 — 6D pose 직접 출력 안 함
- CNN 흑상자 — 디버깅 어려움

5. FALCON-1 적용 결정

결정 항목	값
채택	SHOULD baseline (Sprint 4 검토)
사용	Hand-coded vs GG-CNN 학회 paper 비교
v1.0 production	비채택 (Hand-coded 우월)

6. 참고

- 논문 : Morrison et al. 2018
- GitHub : <https://github.com/dougsm/ggcnn>
- Cornell Grasp dataset :
https://web.archive.org/web/20210127062817/http://pr.cs.cornell.edu/grasping/rect_data/data.php

H3. Contact-GraspNet (NVIDIA, 2021)

BLUF ★★★ v1.1 검토 (BRAVO 한정). NVIDIA 2021 출시 학습 기반 6D grasp prediction, point cloud 입력 → 6D grasp pose + width 직접 출력. 85-92% 성공률, GraspNet-1Billion(H4)으로 SOTA 양보. v1.0 PoC는 Hand-coded(H1) 우월, v1.1 새 도구 추가 시 학습 기반 narrative 강화.

1. 개요

항목	내용
풀네임	Contact-GraspNet
출시	2021년 (ICRA 2021)
라이선스	NVIDIA Source Code License

항목	내용
관리	NVIDIA Research
입력	point cloud (segmented)
출력	6D grasp pose + gripper width + score
학습 데이터	Acronym (NVIDIA, ~17M grasps)
성공률	85-92%
VRAM	약 4 GB

2. FALCON-1 사용 시나리오 (v1.1)

- v1.1 새 공구 (관절형 캘리퍼·전동공구) 추가 시 zero-shot grasp
- FoundationPose(F4) + Contact-GraspNet 결합 — 'AprilTag 없이 grasp' narrative
- 학습 데이터 라벨링 자동화 — Acronym pretrained 모델 활용

3. FALCON-1 적용 결정

결정 항목	값
v1.0 채택	비채택 (Hand-coded 우월)
v1.1 검토	BRAVO 한정 — 새 공구 narrative
주요 의존성	F2 Open3D (point cloud), F4 FoundationPose

4. 참고

- GitHub : https://github.com/NVlabs/contact_graspnet
- 논문 : Sundermeyer et al. ICRA 2021
- Acronym dataset : <https://sites.google.com/nvidia.com/graspdataset>

H4. GraspNet-1Billion

BLUF ★★★ v1.1 SOTA 후보. 1 billion grasp 학습 데이터 + 학습 모델, 92-95% 성공률로 학습 기반 grasp의 SOTA. Contact-GraspNet 후속 위치. v1.1+ 본격 도입 시 narrative 강력 — 'industrial-scale grasp learning'.

1. 개요

항목	내용
출시	2020년 (CVPR)
라이선스	MIT (코드), CC BY-NC (데이터)
관리	Shanghai Jiao Tong University
학습 데이터	1 billion grasps (97 objects)
성공률	92-95% (Reaal Robot)
VRAM	5-6 GB

2. FALCON-1 적용 결정

결정 항목	값
v1.0 채택	비채택
v1.1 검토	Contact-GraspNet과 양자택일 (성공률 우위)
narrative	1B 학습 데이터 키워드 강력

3. 참고

- GitHub : <https://github.com/graspnet/graspnet-baseline>

- 논문 : Fang et al. CVPR 2020
- Dataset : <https://graspnet.net/>

H5. AnyGrasp (2023)

BLUF ★★ v1.2+ 검토. 동적 환경 + temporal smoothness 강화 grasp. 작업자 손이 동시에 같은 영역에 있는 경우(안전 + 협업) 가치. v1.0 정적 stationary work cell에는 overengineering, v1.2+ 다중 작업자 환경 narrative.

1. 개요

항목	내용
풀네임	AnyGrasp
출시	2023년 (T-RO)
라이선스	비상업 (학술용)
관리	Shanghai Jiao Tong University (GraspNet 동일)
특징	Temporal smoothing + dynamic environment
성공률	95%+ (동적 환경)

2. FALCON-1 적용 결정

결정 항목	값
v1.0 채택	비채택 (정적 환경에 overengineering)
v1.2+ 검토	다중 작업자 환경 narrative 시

3. 참고

- GitHub : https://github.com/graspnet/anygrasp_sdk
- 논문 : Fang et al. T-RO 2023

H6. Grasplt! (legacy)

BLUF ★ 비권장 (legacy). 1990년대 출시 grasp 시뮬레이터, 학술 표준이었으나 현재 사실상 EOL. ROS 통합 graspit_interface 있으나 ROS 1만 지원. FALCON-1 적용 안 함, 학습·역사적 참고만.

1. 비권장 사유

15. ROS 2 미지원 — graspit_interface 는 ROS 1 EOL (2025-05)
16. Hand-coded(H1) + Pilz LIN 으로 v1.0 충분
17. 학습 기반 (H3-H5)으로 v1.1+ 진화
18. Grasplt! GUI 도구 — 시뮬 기반, real robot 직접 통합 약함

2. FALCON-1 적용 결정

결정 항목	값
채택	비권장 (legacy)

3. 참고

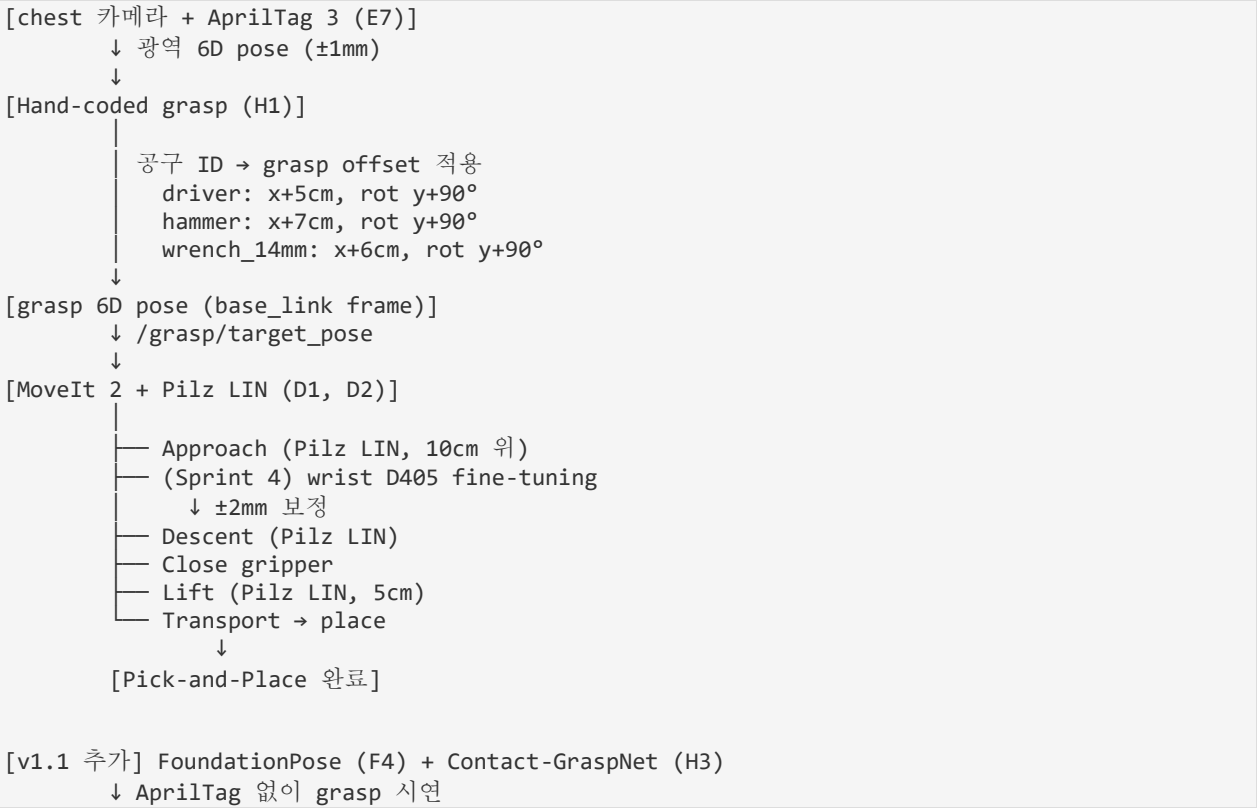
- Grasplt! : <https://graspit-simulator.github.io/>
- 논문 : Miller, Allen 2004

영역 H 종합 결론

1. 채택 / 비채택 정리

#	기술	v1.0 채택	도입 Sprint	역할
H1	Hand-coded grasp	✔MUST	Sprint 3	AprilTag + offset, 4종 공구
H2	GG-CNN	SHOULD baseline	Sprint 4	학회 paper 비교
H3	Contact-GraspNet	v1.1 검토	v1.1	새 공구 narrative
H4	GraspNet-1Billion	v1.1 SOTA	v1.1	1B 데이터 narrative
H5	AnyGrasp	v1.2+	v1.2+	동적 환경
H6	Graspl!	비권장 (legacy)	—	역사적 참고

2. 통합 Grasp Pipeline (v1.0)



3. v1.0 정확도 예산

계층	기여 오차	누적
AprilTag 6D pose	±1 mm (E7)	±1 mm
Hand-coded offset 정의 정확도	±0.5 mm	±1.5 mm
Movelt 2 IK (KDL)	±0.5 mm	±2 mm
Pilz LIN trajectory	±0.5 mm	±2.5 mm
ros2_control 1kHz 정확도	±0.5 mm	±3 mm
wrist D405 fine-tuning (Sprint 4 선택)	-1.5 mm 보정	±1.5 mm 최종

핵심 결론 : Hand-coded + AprilTag만으로 ±2-3mm 정확도 충분. wrist D405 fine-tuning 추가 시 ±1.5mm 도달.

4. Sprint 별 도입 list

Sprint	도입 기술	주요 작업
Sprint 2	AprilTag 부착 + 캘리브레이션	tag36h11 인쇄·부착, 카메라 캘리브레이션
Sprint 3	Hand-coded grasp 4종 정의	공구별 offset hand-tuning, MTC 통합
Sprint 4 (선택)	wrist D405 fine-tuning + GG-CNN baseline	Hand-coded ±2mm → ±1.5mm, 학회 비교
Sprint 5	100회 burn-in 측정	Hand-coded 성공률 ≥ 95% 검증

5. 잔여 위험 및 대응

#	위험	심각도	대응
1	AprilTag 가림 시 grasp 실패	□ Med	공구별 다중 Tag 부착, retry mechanism

#	위험	심각도	대응
2	Hand-coded offset 부정확	□ Low	Sprint 3 hand-tuning 시간 충분 확보
3	공구 형상 변동 (다른 모델)	□ Med	v1.0 4종 고정, v1.1 학습 기반 전환
4	성공률 95% 미달	□ Med	wrist D405 fine-tuning 의무 도입
5	학회 paper narrative 약함 (hand-coded)	□ Low	'AprilTag-anchored baseline' framing + GG-CNN 비교

6. 검증 시험 (Test Plan 매핑)

Test ID	내용	Sprint	성공 기준
T-GRASP-01	Hand-coded grasp 단발 성공률	Sprint 3	≥ 90% (50회)
T-GRASP-02	Hand-coded + fine-tuning 정확도	Sprint 4	TCP 위치 ±2 mm
T-GRASP-03	Burn-in 100회 반복 성공률	Sprint 5	≥ 95% (Hand-coded primary)
T-GRASP-04	Cycle time (grasp ~ place)	Sprint 5	< 30s 평균
T-GRASP-05	GG-CNN 비교 (선택)	Sprint 4	Hand-coded 우월 확인

7. 다음 작성 영역

영역 H Grasp Planning 6종 분석 완료. 후속 영역 :

19. 영역 I — LLM · Reasoning (6 종) : Llama 3.2 1B, Gemma 2 2B, Phi-3.5 mini, Qwen 2.5, Function calling layer, llama.cpp/Ollama

20. 영역 J — Learning RL · Imitation · VLA (8 종)

21. 영역 K — Sim · CI · Logging (8 종)

22. 영역 L — Safety · Power (6 종)

부록 — 변경 이력

버전	일자	내용	작성
v0.1	2026-04-30	영역 H Grasp Planning 6종 초안 (BROWN accent)	이건희, CTO 페르소나